



mirongaskov
5 часов назад

Современный дата-стек: потоковая система из «LEGO»

Средний 17 мин 5.1K

Big Data*, DevOps*, Java*, Распределённые системы*

Тutorial



Вы слышали о **Kafka**, **MQTT**, **S3**, **Iceberg**, **Trino**, **PostgreSQL**, **Redis** и **Flink**? А насколько хорошо вы знаете эти технологии? По каждой из них написаны огромные книги («Kafka: The Definitive Guide», около 800 страниц), и каждый день выходят новые публикации про тонкости.

Эта статья про другое.

Вместо внутренностей движков и законов распределённых систем посмотрим на эти технологии как на кубики LEGO: какую роль каждая из них играет в архитектуре и как они стыкуются друг с другом. Это будет практический tutorial: начнём с минимальной конфигурации и постепенно соберём сложную систему. Статью можно просто читать как обзор архитектуры, а можно запускать каждую конфигурацию и изучать её в деталях. Для этого достаточно **Git**, **Git LFS** и **Docker Compose**. Всё запускается в контейнерах. Даже примеры на **Java** собираются через *Docker multi-stage build*.

Итак, представим, что по всему земному шару распределены датчики, которые с некоторой периодичностью отправляют свои координаты и температуру. Будем собирать, хранить и анализировать эти показания в потоковом режиме.

Платформа: **Kafka**, **Kafka Connect**, **Schema Registry**, **Kafka UI**

Для tutorials нужен конструктор <https://github.com/IoTDataPlatform/constructor>. Перед клонированием репозитория установите **Git LFS**.

```
git clone https://github.com/IoTDataPlatform/constructor.git
cd constructor
```

Объяснить с  SourceCraft

Сейчас файлов немного:

```
tree
├── bricks
│   ├── docker-compose.yml
│   ├── Dockerfile
│   ├── filter_tags.py
│   └── fs.tar.gz
└── README.md
```

Объяснить с 

В директории **bricks** лежат «кубики» системы и конструктор на **Python**. В архив **fs.tar.gz** упакованы исходники, некоторые **Java**-библиотеки из репозитория **Confluent** и **JAR**-файлы коннекторов. Прошу прощения за размер архива, но так сборка примеров для tutorials работает очень быстро. Да и вообще, некоторые **JAR** невозможно скачать напрямую из РФ из-за взаимных блокировок. Запускаем конструктор:

```
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg)
```

Объяснить с 

▸ [Как работает конструктор](#)

Конструктор добавил новые файлы и директории — это и есть код системы:

```
tree
├── docker-compose.yml
├── kafka-connect
│   ├── connectors-pack.tar.gz
│   ├── Dockerfile
│   ├── init
│   │   └── init.sh
│   └── sources
```

Объяснить с 

Центральный файл **docker-compose.yml** описывает запуск компонентов:

- **Kafka**, распределённая потоковая платформа;
- **Kafka Connect**, сервис переноса данных между **Kafka** и другими системами;
- **Confluent Schema Registry**, реестр схем сообщений;

- **Kafka UI**, веб-интерфейс для работы со всеми этими компонентами.

Назначение остальных файлов изучим по ходу дела. А пока запустим:

```
docker compose up -d --build
```

Объяснить с 

После старта всех сервисов по ссылке <http://localhost:8080> откроется такой интерфейс **Kafka UI**:

Стартовая страница Kafka UI

Ingest: пуллинг через Kafka Connect

Сейчас в нашей системе нет данных. Будем это исправлять, добавляя компоненты. По зоне ответственности компоненты можно разделить на четыре группы. *Ingest* доставляет данные из внешнего мира в систему. *Storage* хранит их персистентно. *Transform* преобразует данные в удобные производные. А *Serve* делает их доступными для внешних систем.

Подтянем данные в нашу систему методом *polling*, как будто сама система периодически опрашивает датчики температуры. Реальных сенсоров у нас нет, поэтому будем генерировать синтетические показатели. Откатим, пересоберём конструктор и запустим систему:

```
docker compose down -v
git clean -fd
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags SOURCE)
docker compose up -d --build
```

Объяснить с 

▸ [Немного деталей](#)

Сразу же идём на **Kafka UI** <http://localhost:8080> и переходим в раздел *Topics*:

Список топиков, включая системные

Думая о данных, мы обычно представляем таблицы и связи между ними. Таблицы отражают состояние объектов. В потоковых системах фокус смещается с состояния на события — моменты времени, когда что-то произошло или изменилось.

Так что же у нас вместо таблиц? В **Kafka** базовая абстракция хранения и обработки данных называется *топиком*. Через *топики* приложения обмениваются данными. Вместо строк или записей в классических базах данных *топики* содержат *сообщения*. Новые *сообщения* дописываются в конец *лога* и после этого уже не изменяются. *Топики*, названия которых начинаются с двух подчёркиваний, являются внутренними системными. Их мы изучать не будем. Сейчас нас интересует топик **earth-temp**. Заходим в него и раскрываем первое сообщение в списке:

Сообщения топика earth-temp

Key — идентификатор датчика. **Value** выглядит как обычный **JSON**.

```
{
  "sensorId": {"string": "zzsvd7rr3rdo2p8ybjwguvqex"},
  "latitude": {"double": 16.17987628543722},
  "longitude": {"double": 25.77224771279677},
  "timestamp": {"long": "2026-04-12T03:35:27.035Z"},
  "temperatureF": {"double": 63.6}
}
```

[Объяснить с](#) 

Однако обратите внимание, что размер **value** всего лишь 66 байт. Но длина в символах явно больше! Это потому, что сообщение закодировано в **Avro**, эффективном бинарном формате. **Kafka** безразлично, что именно лежит внутри сообщения. Для неё что ключ, что значение всего лишь последовательность байтов, которые могут содержать **JSON**, **CSV**, **HTML**, бинарные данные и что угодно ещё. Именно взаимодействующие приложения должны договориться о формате (контракте). Но если это просто массив байтов, который даже не содержит названий полей, то как **Kafka UI** отображает сообщения в читаемом виде? Чтобы декодировать **Avro**, нужна схема, с помощью которой это сообщение было *сериализовано*, так называемая **writer schema**. А эта схема должна где-то храниться. Вспоминаем про **Schema Registry** и открываем его на **UI**:

Confluent Schema Registry

Там лежит схема сообщений для *топики* **earth-temp**. Сообщения хранят **schema ID**, что позволяет приложениям правильно декодировать данные из массива байтов. Обратите внимание также на свойство **Compatibility BACKWARD**. Реестр не просто хранит схемы сообщений, но и проверяет их на совместимость. Сейчас в эту тему углубляться не будем.

Мы посмотрели на сообщения, посмотрели на их схему, но откуда сообщения появляются? Заходим в раздел **Kafka Connect**:

Коннекторы

Приложения, которые пишут данные в **Kafka**, называются **producer**'ами. Сейчас таким *продюсером* является *коннектор* типа **SOURCE**. По легенде он опрашивает датчики и записывает их показания. По мере развития системы появятся и другие *коннекторы*, а через этот **UI** можно за ними наблюдать, менять конфигурацию и перезапускать:

Конфигурация пулл-коннектора

Чтобы всё это заработало, конструктор добавил такие файлы:

```
tree
├── docker-compose.yml
├── kafka-connect
│   ├── connectors-pack.tar.gz
│   ├── Dockerfile
│   ├── init
│   │   ├── earth-temp-connect-source.json
│   │   └── init.sh
│   └── sources
│       ├── temp-source-connector
│       │   ├── build.gradle
│       │   ├── settings.gradle
│       │   └── src
│       │       ├── main
│       │       │   ├── java
│       │       │   │   ├── dev
│       │       │   │   │   ├── miron
│       │       │   │   │   │   ├── connect
│       │       │   │   │   │   │   ├── temp
│       │       │   │   │   │   │   │   ├── TempSourceConnector.java
│       │       │   │   │   │   │   │   └── TempSourceTask.java
│       │       │   └── resources
│       │       │       ├── META-INF
│       │       │       └── services
│       │       │           └── org.apache.kafka.connect.connector.Connector
```

[Объяснить с !\[\]\(feabb98897b440bc8695a03336a6e2df_img.jpg\)](#)

Плагин для **Kafka Connect**, который генерирует температуру и координаты, реализован на **Java** в директории **sources**. Сборка происходит в **Docker** в *multi-stage build*:

```
FROM gradle:8.7-jdk17 AS builder
```

```
WORKDIR /src
```

```
COPY sources/temp-source-connector/ ./

RUN gradle --no-daemon clean jar

FROM confluentinc/cp-kafka-connect-base:8.1.0

ADD connectors-pack.tar.gz /usr/share/confluent-hub-components/

COPY --from=builder /src/build/libs/temp-source-connector-1.0.0.jar /usr/share/confluent-hub-components/
```

[Объяснить с](#) 

Кроме этого *коннектора* во время сборки устанавливается и набор стандартных из архива **connectors-pack.tar.gz**. Их мы будем использовать в следующих разделах статьи. **init** — самая интересная директория, которая будет пополняться файлами по мере развития системы. Здесь собраны конфигурации всех *коннекторов*, а скрипт **init.sh** запускает их через **REST API Kafka Connect**:

```
#!/bin/sh
set -eu

for file in /init/*.json; do
    [ -e "$file" ] || exit 0

    name="$(basename "$file" .json)"
    echo "Applying connector: $name"

    curl -fsS -X PUT \
        "$CONNECT_URL/connectors/$name/config" \
        -H "Content-Type: application/json" \
        --data @"$file"

    echo
done

echo "Init done"
```

[Объяснить с](#) 

Сейчас здесь только одна конфигурация:

```
{
  "connector.class": "dev.miron.connect.temp.TempSourceConnector",
  "tasks.max": "1",
  "topic": "earth-temp",
  "sensors": "5",
  "messages.per.sensor.per.second": "1"
}
```

[Объяснить с](#) 

В этом файле задано, сколько событий генерировать и в какой *топик* записывать. При старте по земному шару распределяются случайным образом датчики, а потом с заданной периодичностью

генерируются показания температуры с учётом удалённости от экватора и времени дня. В **docker-compose.yml** описан сервис **kafka-connect** и *one-shot* **connect-init**:

```
kafka-connect:
  build:
    context: kafka-connect
  image: my-cp-kafka-connect:8.1.0
  container_name: kafka-connect
  hostname: kafka-connect

connect-init:
  image: alpine/curl:latest
  depends_on:
    kafka-connect:
      condition: service_healthy
  volumes:
    - ./kafka-connect/init:/init:ro
  environment:
    CONNECT_URL: http://kafka-connect:8083
  entrypoint: ["/bin/sh", "/init/init.sh"]
  restart: "no"
```

[Объяснить с](#) 

Storage: реляционная база данных PostgreSQL

Kafka сама может хранить данные с настраиваемым периодом хранения. Но для аналитических запросов она подходит плохо. Добавим **PostgreSQL**:

```
docker compose down -v
git clean -fd
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags SOURCE,POSTG
docker compose up -d --build
```

Список *коннекторов* <http://localhost:8080/ui/clusters/local/kafka-connect/connectors?fts=false>:

Два коннектора типа SOURCE и SINK

Появился *коннектор* типа **SINK**. Его задача — подхватывать сообщения из топика **earth-temp** и отправлять в базу данных на постоянное хранение. Ещё одно важное изменение можно увидеть на **UI** в разделе **Consumers** <http://localhost:8080/ui/clusters/local/consumer-groups/connect-earth-temp-postgres-sink>:

PostgreSQL SINK Consumer

Consumer — приложение, которое читает сообщения из *топиков*. *Топик* в **Kafka** — это именно *лог*, а не очередь. Когда **consumer** прочитывает сообщение, оно из-за этого не удаляется. Другой **consumer** сможет прочитать то же сообщение ещё раз позднее. **Consumer group** — это один или несколько процессов, которые делят между собой обработку сообщений в *топике*. На один *топик* может существовать несколько таких **consumer group**, которые выполняют разные задачи. **Kafka** хранит позицию чтения — **offset**. Поэтому если какой-нибудь читающий процесс упадёт, после перезагрузки он сможет продолжить с первого необработанного сообщения. **Offset** фиксируется в результате **commit**, когда **consumer** записывает свой прогресс. На странице выше можно отслеживать отставание (**Consumer Lag**) от конца *топика*.

Теперь структура такая:

```
tree
├── kafka-connect
│   ├── connectors-pack.tar.gz
│   ├── Dockerfile
│   ├── init
│   │   ├── earth-temp-connect-source.json
│   │   ├── earth-temp-postgres-sink.json
│   │   └── init.sh
├── postgres
│   ├── init
│   └── init.sql
```

Объяснить с 

В конфигурации *коннектора* задано, из какого *топики* и в какую таблицу какой базы писать:

```
{
  "connector.class": "io.confluent.connect.jdbc.JdbcSinkConnector",
  "tasks.max": "1",
  "topics": "earth-temp",
  "connection.url": "jdbc:postgresql://postgres:5432/weather",
  "connection.user": "postgres",
  "connection.password": "postgres",
  "table.name.format": "earth_temp",
  "insert.mode": "insert",
  "auto.create": "false",
  "auto.evolve": "false",
  "quote.sql.identifiers": "always"
}
```

Объяснить с 

Скрипт создания таблицы:

```
create table if not exists earth_temp (
  id bigserial primary key,
  "sensorId" text,
  "latitude" double precision,
  "longitude" double precision,
  "timestamp" timestamptz,
  "temperatureF" double precision,
  "created_at" timestamptz not null default NOW()
);
```

Объяснить с 

И сервис в **docker-compose.yml**:

```
postgres:
  image: postgres:16-alpine
  container_name: postgres
  hostname: postgres
  environment:
```

```
POSTGRES_USER: postgres
POSTGRES_PASSWORD: postgres
POSTGRES_DB: weather

ports:
  - "5432:5432"

volumes:
  - postgres-data:/var/lib/postgresql/data
  - ./postgres/init:/docker-entrypoint-initdb.d:ro
```

[Объяснить с](#) 

PostgreSQL здесь играет не только роль *Storage*, но и роль *Serve*. Приложения могут запрашивать данные через **SQL**. Выполним запрос через **DataGrip**:

Подключение к PostgreSQL

Отправляем SQL-запрос

Storage: Lakehouse на S3 + Iceberg

Есть и другой распространённый подход к хранению аналитических данных: **Data Lake**, или *озеро данных*. В объектном хранилище типа **S3** лежат файлы произвольного формата, например, **JSON**, **Parquet**, **CSV**, **Avro**. А могут быть и вовсе неструктурированными. Такой подход позволяет хранить огромный объём данных. Он также позволяет отделить хранение от обработки. Однако, без контроля **Data Lake** легко превращается в **Data Swamp**, или *болото данных*. Поэтому часто вводят дополнительный уровень со структурой хранения и схемами. Такой подход называется **Lakehouse** и представляет собой нечто среднее между **Data Lake** и **Data Warehouse**. Данные по-прежнему лежат в объектном хранилище, но уже в упорядоченном виде. Вычисления отделены от хранения. **Apache Iceberg** — открытый табличный формат для такого подхода. Заменяем **PostgreSQL** на **Lakehouse**:

```
docker compose down -v
git clean -fd
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags SOURCE, ICEBERG
docker compose up -d --build
```

Объяснить с 

Коннектор теперь другой <http://localhost:8080/ui/clusters/local/kafka-connect/connectors?fts=false>:

Iceberg SINK коннектор

В системе появился сервис **MinIO** с интерфейсом <http://localhost:9001/login>:

Страница входа в MinIO

Это **S3**-совместимое хранилище, и визуально оно похоже на файловую систему. Объекты хранятся в **бакетах** — условно логических дисках:

Бакет weather

Спускаемся `earth/temp_raw/data/timestamp_day=2026-04-12/` и видим **Parquet**-файлы:

Parquet файлы

Колоночный формат **Parquet** хорошо подходит для аналитических данных. Можно хранить широкие денормализованные таблицы, а потом быстро считывать только нужные для аналитического запроса колонки. Такая оптимизация называется **projection**. Формат значительно отличается от *В-деревьев*, которые используют в реляционных базах данных, которые лучше подходят для транзакционной нагрузки.

Файлы лежат в директории `timestamp_day=2026-04-12`. Формально это не совсем корректно. Хранилища типа **S3** хранят данные как пары ключ-значение. Корректнее говорить не о пути к файлу, а о префиксе ключа объекта. Не будем жертвовать понятностью ради формальной точности. Можно сказать, что данные *партиционированы* по дате события. Так, если нам нужна аналитика за определённый период, мы быстро выберем нужные данные. Такая оптимизация называется **partition pruning**.

Данные разбиты по календарным дням

По пути к данным мы пропустили "директорию" **metadata**. Именно метаданные делают **Iceberg** табличным форматом. Зачем вообще понадобился этот слой? Мы могли бы просто хранить **Parquet**-файлы в *бакетах*. Можно было бы дописывать данные в эти файлы или просто добавлять новые. Но если задуматься над тем, как организовать удаление записей, частичные обновления, что делать во время поломки сети с незавершёнными записями, то всё становится сложнее. А если учесть, что нам надо поддерживать конкурентную запись разными системами, то необходимость табличного слоя становится очевидной. **Iceberg** определяет то, как множество файлов объединяется в одну логическую таблицу.

Данные и метаданные

Serve: федеративный движок запросов Trino

Для работы с данными и метаданными **Iceberg** есть специальные библиотеки. Но не будем же мы писать низкоуровневый код на **Java** для каждого аналитического запроса? С **PostgreSQL** было явно удобнее. А если запросы окажутся тяжёлыми? Нам придётся писать код для распределённого исполнения? К счастью, для **Lakehouse** уже есть распределённые **SQL**-движки. Помните, мы говорили про разделение **storage** и **compute**. В нашей конфигурации **Trino** и будет **compute**-частью этого разделения. Он *федеративный*, потому что может выполнять запросы не только к **Iceberg**, но и ко многим другим системам хранения, даже в рамках одного запроса.

```
docker compose down -v
git clean -fd
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags SOURCE,ICEBERG
docker compose up -d --build
```

[Объяснить с](#) 

Подключимся к нему из **DataGrip** и выполним запрос:

Подключаемся к Trino

Выполняем запрос

В итоге схема такая: данные лежат в объектном хранилище в формате **Parquet**, **Iceberg** держит над ними метаданные табличного слоя, **Trino** читает таблицу, а снаружи всё выглядит как обычный **SQL**.

Код системы организован сейчас так:

```
tree
├── docker-compose.yml
├── kafka-connect
│   ├── connectors-pack.tar.gz
│   ├── Dockerfile
│   ├── init
│   │   ├── earth-temp-connect-source.json
│   │   ├── earth-temp-iceberg-sink.json
│   │   └── init.sh
│   └── sources
│       └── temp-source-connector
│           └── ...
└── trino
    ├── catalog
    └── iceberg.properties
```

Объяснить с 

Ничего особо нового. Всё те же *коннекторы* и настройки **Trino**.

Transform: обогащение сообщений с помощью Kafka Streams

До этого момента мы просто передавали данные между системами. В потоковой обработке (**stream processing**) сообщения обрабатываются по мере их поступления. Накопления, как в **batch processing**, здесь не происходит. Потоковую обработку можно разделить на два вида: без состояния (**stateless**) и с состоянием (**stateful**).

В нашем примере для аналитики полезно иметь в сообщениях температуру в градусах Цельсия, часовой пояс, локальное время и индекс НЗ. Для такого обогащения явно не нужно никакое состояние, так как каждое событие не зависит от других. Это трансформация типа **map**. Реализовать её можно несколькими способами. Например, можно написать программу на **Java** и через низкоуровневый **Kafka API**: в цикле вычитывать сообщения из *топика*, формировать новые расширенные сообщения и отправлять их в другой *топик*. Но удобнее взять более высокоуровневую библиотеку **Kafka Streams**. С её помощью такие программы можно писать в относительно декларативном стиле.

```
docker compose down -v
git clean -fd
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags SOURCE,ICEBERG
```

Объяснить с 

Появилась директория с кодом:

```
tree
├── streams
│   ├── Dockerfile
│   ├── repo-confluent.tar.gz
│   ├── sources
│   │   └── temp-enricher
│   │       ├── build.gradle
│   │       ├── settings.gradle
│   │       └── src
│   │           ├── main
│   │           │   ├── java
│   │           │   │   ├── dev
│   │           │   │   │   ├── miron
│   │           │   │   │   │   ├── streams
│   │           │   │   │   │   │   ├── temp
│   │           │   │   │   │   │   └── TempEnricherApp.java
│   └── temp-enricher.properties
```

Объяснить с 

Отрывок кода в относительно декларативном стиле:

```
StreamsBuilder builder = new StreamsBuilder();
builder.stream(inputTopic, Consumed.with(keySerde, valueSerde))
    .mapValues(value -> enrich(value, h3Resolution))
    .to(outputTopic, Produced.with(keySerde, valueSerde));
```

Объяснить с 

И ещё примерно 300 строк написаны уже не в декларативном стиле.

Запустим:

```
docker compose up -d --build
```

Объяснить с 

И получим ещё один топик **earth-temp-enriched** <http://localhost:8080/ui/clusters/local/all-topics/earth-temp-enriched/messages?valueSerde=SchemaRegistry&limit=100&mode=LATEST>

Обогащённое сообщение

Этот *топик* мы тоже через *коннектор* перегоняем в таблицу **Iceberg**. Запрос к ней:

Запрос к обогащённой таблице через Trino

Transform: аналитический процессинг на Flink

В прошлой части подход получился не таким уж декларативным. А если бы пришлось реализовывать более сложную трансформацию с сохранением состояния, учётом течения времени и запаздывающих событий, то было бы ещё сложнее. **Flink** позволяет реализовывать сложные трансформации на языке **SQL**. Более декларативный подход придумать трудно.

```
docker compose down -v
git clean -fd
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags SOURCE,FLINK
```

[Объяснить с !\[\]\(d0a1791f26d167e866e44ebbf83efebe_img.jpg\)](#)

В коде системы появились сервисы для **Flink** и скрипт трансформации:

```
tree
├── flink
│   ├── Dockerfile
│   ├── flink-libs.tar.gz
│   └── init
│       └── temp-avg-1min.sql
```

[Объяснить с !\[\]\(eafc244b53721dd1ec133f0772f70fc7_img.jpg\)](#)

В этой трансформации вычисляются средние значения по минутным окнам. А код в полностью декларативном стиле описывает это: подключение к *топику*, отбор сообщений с полной информацией, расчёт средних по минутным окнам и запись результата во второй топик:

```
CREATE TABLE temp_raw
(
    sensorId      STRING NULL,
    `timestamp`   TIMESTAMP(3) NULL,
    latitude      DOUBLE NULL,
    longitude     DOUBLE NULL,
    temperatureF  DOUBLE NULL,
    WATERMARK FOR `timestamp` AS `timestamp` - INTERVAL '5' SECOND
```

```

) WITH (
    'connector' = 'kafka',
    'topic' = 'earth-temp',
    'properties.bootstrap.servers' = 'kafka:9092',
    'properties.group.id' = 'flink-temp',
    'scan.startup.mode' = 'group-offsets',
    'properties.auto.offset.reset' = 'earliest',
    'format' = 'avro-confluent',
    'avro-confluent.schema-registry.url' = 'http://kafka-schema-registry:8081'
);

CREATE VIEW temp_strict AS
SELECT
    sensorId,
    `timestamp`,
    latitude,
    longitude,
    temperatureF
FROM temp_raw
WHERE sensorId IS NOT NULL
    AND `timestamp` IS NOT NULL
    AND latitude IS NOT NULL
    AND longitude IS NOT NULL
    AND temperatureF IS NOT NULL;

CREATE TABLE temp_avg (
    sensorId      STRING NOT NULL,
    window_start  TIMESTAMP(3) NOT NULL,
    window_end    TIMESTAMP(3) NOT NULL,
    latitude       DOUBLE NOT NULL,
    longitude      DOUBLE NOT NULL,
    temperatureF  DOUBLE NOT NULL
) WITH (
    'connector' = 'kafka',
    'topic' = 'earth-temp-avg',
    'properties.bootstrap.servers' = 'kafka:9092',

    'key.format' = 'raw',
    'key.fields' = 'sensorId',

    'format' = 'avro-confluent',
    'avro-confluent.schema-registry.url' = 'http://kafka-schema-registry:8081'
);

INSERT INTO temp_avg
SELECT
    sensorId,
    window_start,
    window_end,
    AVG(latitude)      AS latitude,
    AVG(longitude)     AS longitude,
    AVG(temperatureF) AS temperatureF
FROM TABLE(
    TUMBLE(TABLE temp_strict, DESCRIPTOR(`timestamp`), INTERVAL '1' MINUTE)
)
GROUP BY sensorId, window_start, window_end;

```

Запустим:

```
docker compose up -d --build
```

Объяснить с 

И через некоторое время появится *топик* **earth-temp-avg** с такими сообщениями:

Топик с минутными средними значениями

```
{
  "sensorId": "ca67911wocgu23dp169a7z9ta",
  "window_start": "2026-04-12T07:06:00Z",
  "window_end": "2026-04-12T07:07:00Z",
  "latitude": 0.7856400886251921,
  "longitude": 113.54167794725345,
  "temperatureF": 83.80999999999999
}
```

Объяснить с 

Кластером **Flink** можно управлять здесь <http://localhost:8180>:

UI к Flink кластеру

В отличие от приложения на **Kafka Streams**, **Flink** является полноценной распределённой системой для обработки потоков данных.

Serve: кэш в памяти на Redis

На прошлом шаге мы обошлись без персистентного хранилища. Из кубиков LEGO можно сложить и такую архитектуру. Но для завершённости не хватает компонента *Serve*, чтобы приложения могли получить актуальные данные о средних температурах. Для дашборда, например, нам не нужна история, а нужны данные, которые отражают ситуацию здесь и сейчас. Добавляем компонент **Redis**:

```
docker compose down -v
git clean -fd
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags SOURCE,FLINK
```

[Объяснить с !\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\)](#)

Redis будет хранить для датчиков их последние средние значения в памяти. На этом примере хорошо видна отличительная особенность потоковых систем. В классическом случае мы сохраняли бы все показания в БД, а при запросе пользователя отправляли бы туда **SQL**-запрос. В случае потоковой системы каждое поступающее значение сразу же участвует в расчёте, и как только минутный интервал заканчивается, результат вычисления отправляется дальше по потоку. В нашем случае — в **Redis**.

При такой архитектуре можно быстро менять **SQL** во **Flink** или добавлять новые расчёты и сразу получать в **Redis** новые данные, на которых можно строить обновлённые или новые отчёты. С персистентным хранилищем пришлось бы решать, что делать с уже накопленными данными: удалять их или пересчитывать их. При этом нельзя сказать, что история пропадает. **Kafka** всё равно сохраняет *лог* сообщений с настраиваемой глубиной хранения. И при пересчёте отчёта их можно учесть.

Реализовано всё так же. Дополнительный *коннектор* и сервис в **docker-compose.yml**. Запустим систему:

```
docker compose up -d --build
```

[Объяснить с !\[\]\(0f848bbd71cef6b345273b16f905912a_img.jpg\)](#)

И подключимся к **Redis** через **DataGrip**:

Подключение к Redis через DataGrip

Посмотрим доступные ключи и их значения:

Ключи в Redis

Ingest: брокер сообщений для устройств MQTT

Опрос датчиков — сомнительный вариант **ingest**-модели **pull**. Обычно датчики сами отправляют свою телеметрию — это **ingest**-модель **push**. Теоретически датчик мог бы выступить как **producer** и отправлять сообщения напрямую в **Kafka**. Однако протокол **Kafka** слишком тяжёл для устройств и требователен к качеству связи. Для **IoT** чаще используют более простой протокол **MQTT**. Сервисы, которые реализуют этот протокол, называют **MQTT**-брокерами. В отличие от **Kafka** они не рассчитаны на надёжное хранение, распределённую обработку сообщений и возможность повторного чтения. Их задача проще: принять сообщение от устройства и доставить его подписчикам (модель **publisher/subscriber**). Зато с такими брокерами датчики могут работать в условиях плохой связи, малого объёма трафика и жёстких ограничений по энергии. Связка **MQTT** и **Kafka** получается устойчивой и практичной. Попросим конструктор добавить **MQTT**-брокер и эмулятор датчиков, а потом сразу всё запустим:

```
docker compose down -v
git clean -fd
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags MQTT,FLINK,F
docker compose up -d --build
```

Конструктор добавил **MQTT-коннектор** и код эмулятора на языке **Python**:

```
tree
├── kafka-connect
│   ├── connectors-pack.tar.gz
│   ├── Dockerfile
│   ├── init
│   │   ├── earth-temp-avg-redis-sink.json
│   │   ├── earth-temp-mqtt-source.json
│   │   ├── earth-temp-redis-sink.json
│   │   └── init.sh
│   └── sources
└── mqtt
    ├── mosquitto.conf
    └── temp-sensor-emulator
        ├── app.py
        ├── Dockerfile
        └── requirements.txt
```

Объяснить с 

В **docker-compose.yml** кроме **MQTT-брокера eclipse-mosquitto** и эмулятора датчиков был добавлен и **UI**, доступный по адресу <http://localhost:8086>:

Web UI к MQTT

Подключимся к брокеру:

Подключение к брокеру

Датчики отправляют сообщения в *топики MQTT*. Чтобы их получать, надо на них подписаться. В нашем случае каждый датчик отправляет сообщения в свой собственный *топик* с именем **sensors/{sensorId}/telemetry**. Подпишемся на все *топики* такого вида:

Подписываемся на топики сенсоров

Теперь в UI видны все поступающие сообщения:

Сообщения от датчиков с температурой и координатами

Всё вместе

Здесь не хватает одной стрелки. Кто найдёт, тому респект

А теперь запустим все системы:

```
docker compose down -v
git clean -fd
(cd bricks && docker compose run --rm --user "$(id -u):$(id -g)" lg --tags MQTT,SOURCE,
docker compose up -d --build
```

[Объяснить с](#) 

Вся интеграция завязана на *коннекторы*, поэтому кубики LEGO так легко переставлять местами:

Все коннекторы

Теги: [Kafka](#), [Flink](#), [Redis](#), [Iceberg](#), [Java](#), [Docker](#), [docker-compse](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронпочта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



2

Карма

Мирон Гаськов @mirongaskov

Пользователь

Подписаться



Хабр Карьера

Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для бэкендеров

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

Перейти в поток Бэкенд

Комментарии 3

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K

+96

30

27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой

11 мин

26K

Обзор

Перевод

 +70

 45

 82



prohronus

18 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

 5 мин

 9.3K

Кейс

 +51

 9

 9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

 Простой

 7 мин

 12K

Обзор

 +37

 59

 18



beget_com

23 часа назад

Механический калькулятор. Как работает арифмометр?

 Средний

 10 мин

 9.5K

Обзор

 +37

 23

 8



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

 5 мин

 8.7K

Обзор

 +27

 10

 0



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

 Средний

 14 мин

 9.9K

Мнение

 +26

 33

 11



mariklolik

18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

 Средний

 7 мин

 7.4K

Кейс

+23

23

6



nick_dyuba

22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TipiTip

5 мин 7K

Recovery Mode

+22

8

5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

11 мин 6.9K

+17

9

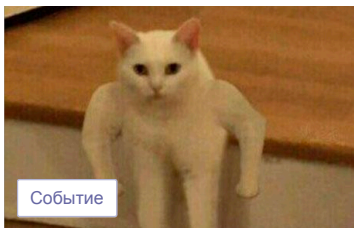
7

Тысяча нюансов: как управлять производством лекарств

Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



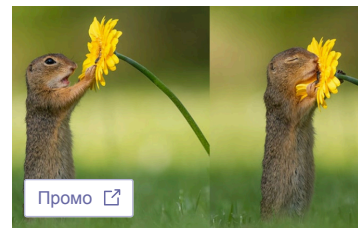
Событие

Качаем к лету хард- и софт-скиллы на айтишных ивентах



Турбо

Удалёнка в фарме: большие данные и никакой магии



Промо

Запахло весной и скидками в Промокодусе

ВОПРОСЫ И ОТВЕТЫ

Выбор брокеров сообщений в зависимости от задачи?

Redis · Простой · 1 ответ

Есть ли готовые решения для интеграции Битрикс24 и брокера сообщений?

RabbitMQ · Простой · 2 ответа

Как переделать этот SQL запрос на java?

Java · Простой · 1 ответ

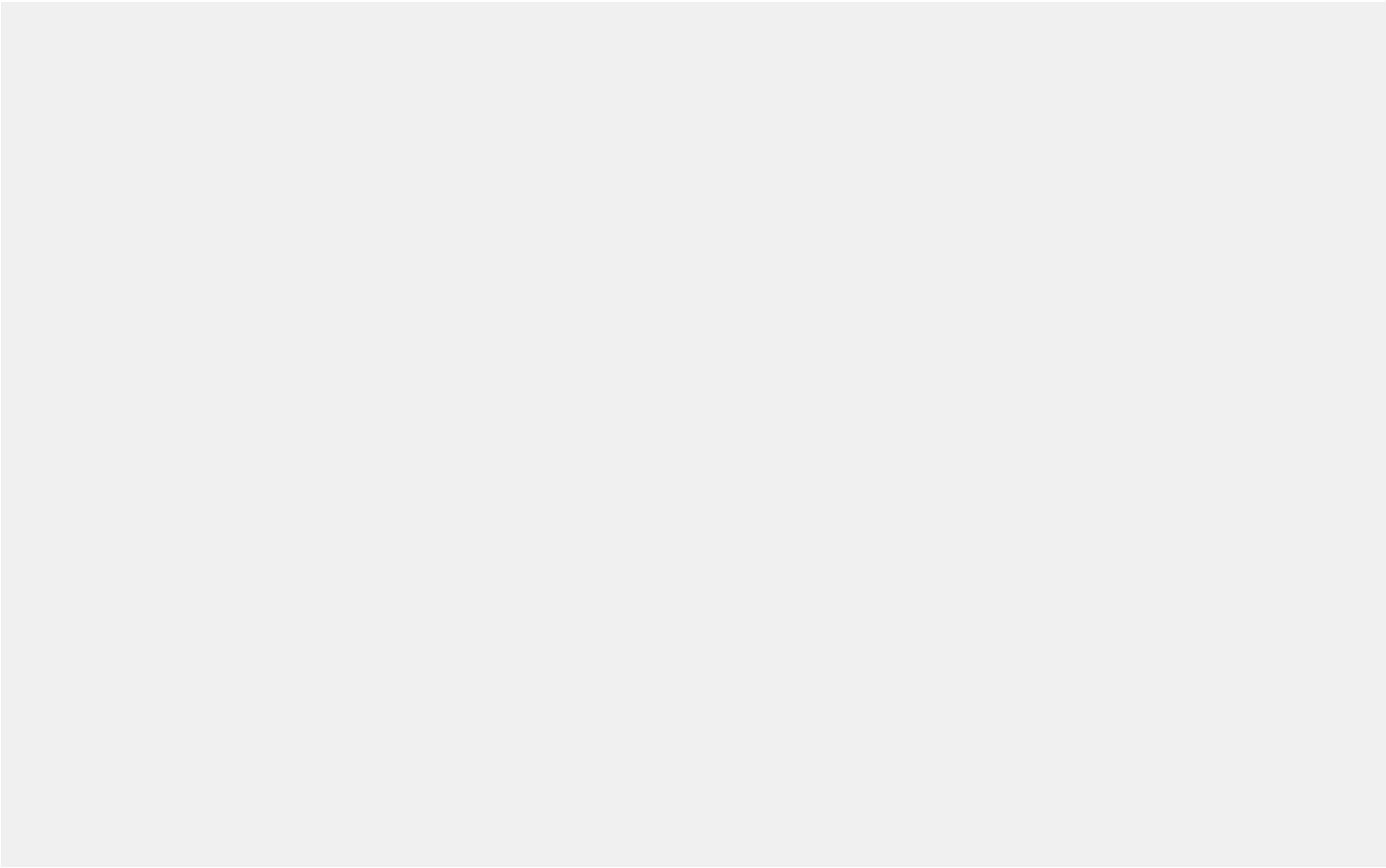
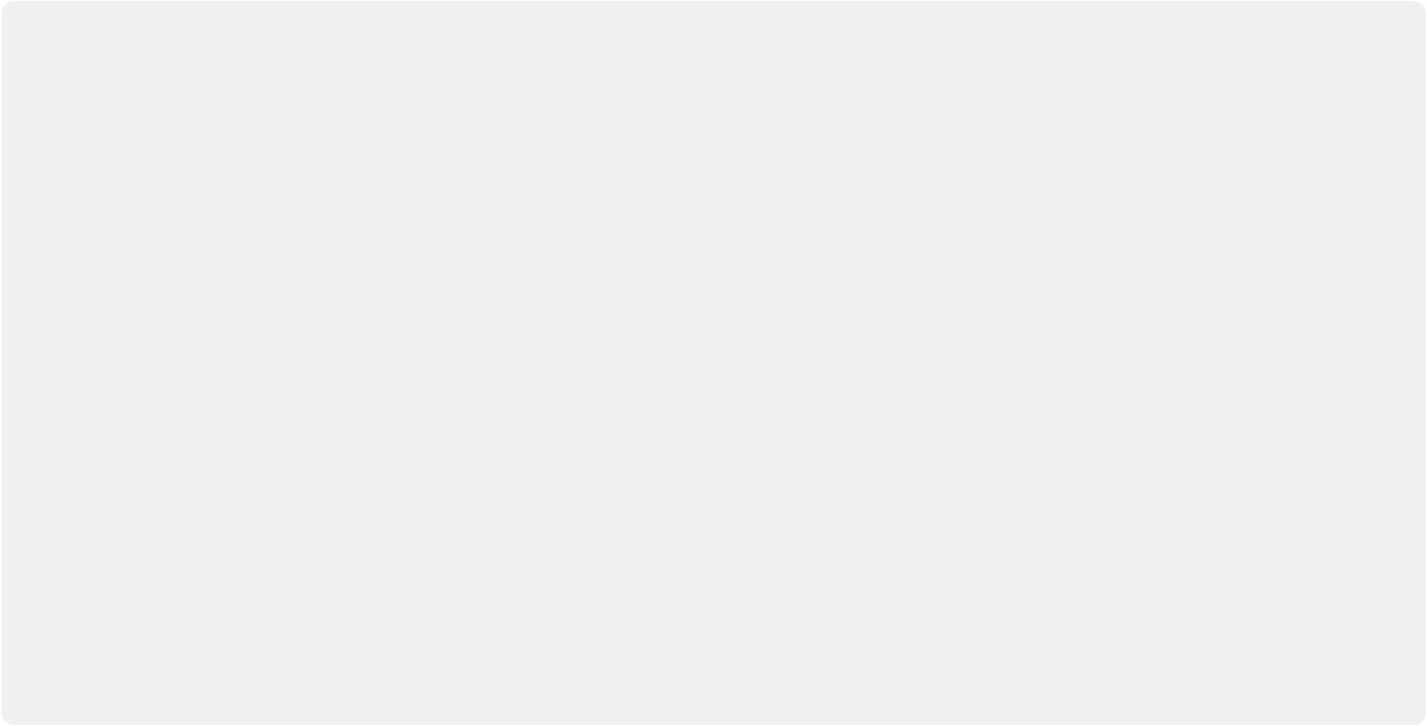
Почему не запускается собранный в .jar файл Java FX проект?

Java · Простой · 2 ответа

Как лучше запустить базу данных Redis?

Redis · Простой · 0 ответов

[Больше вопросов на Хабр Q&A](#)



ЧИТАЮТ СЕЙЧАС

Электроинструмент становится хуже, и это делается намеренно

 26K  82 

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 13K  14 

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 6.7K  15 

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

 5.3K  11 

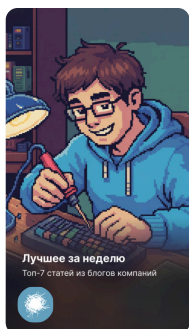
+185% за 13 часов: как Kimi K2.6 переписала 8-летний движок

 14K  9 

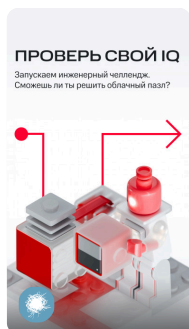
Тысяча нюансов: как управлять производством лекарств

Турбо

ИСТОРИИ



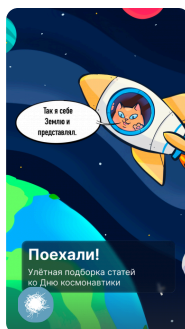
Годнота из блогов компаний



Только 10% решат эту головоломку



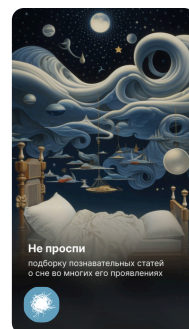
Хочу 450K



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

БЛИЖАЙШИЕ СОБЫТИЯ



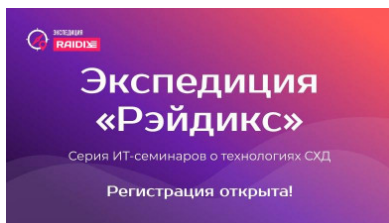
2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная
премия, учрежденная в
2000 году**

Санкт-Петербург

Маркетинг

[Больше событий в календаре](#)



3 марта – 9 июня

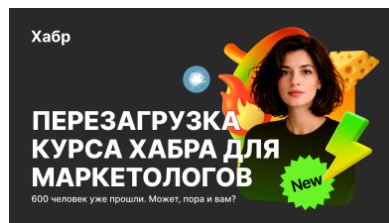
Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск •
Краснодар • Минск • Калининград •
Ростов-на-Дону

Разработка

Другое

[Больше событий в календаре](#)



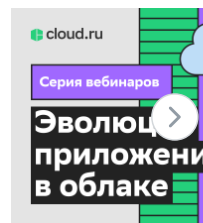
9 марта – 1 июня

**Всё, что нужно, чтобы
стартовать продвижение
бренда на Хабре**

Онлайн

Маркетинг

[Больше событий в календаре](#)



19 марта – 28 апреля

**Серия вебинаров
«Эволюция п
в облаке»**

Онлайн

Разработка

[Больше событий в к](#)

1

2

3

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr